PostgreSQL Commands — Basics → Intermediate → Advanced → Expert (Deep Dive)

A progressive, command-focused guide to help you become deeply proficient with PostgreSQL—from first connection to performance tuning, partitioning, replication, and more. Use it as a *learning roadmap* and a *reference cheat sheet*.

0) Quick Start & `psql` Essentials

Connect

```
# Connect with psql (interactive shell)
psql -h <host> -p <port> -U <user> -d <database>
# Local default socket (common on Linux)
psql -U postgres
# Environment variables
export PGHOST=localhost PGPORT=5432 PGUSER=postgres PGDATABASE=postgres
psql
```

`psql` meta-commands (run inside psql)

```
\?                  -- help for psql
\h <command>       -- SQL help, e.g., \h SELECT, \h CREATE TABLE
\l                  -- list databases
\c <db>            -- connect to database
\dt [schema.]pattern  -- list tables
\dn                -- list schemas
\dv                -- list views
\df [pattern]      -- list functions (\df+, with details)
\du                -- list roles
\d <name>          -- describe table/view/sequence (\d+ shows storage & options)
\x                 -- expand mode on/off (great for wide rows)
\timing            -- toggle query timing
\pset pager off    -- disable pager (or \pset format aligned/unaligned)
\copy ...          -- client-side CSV import/export (uses client file path)
\watch 2           -- rerun the last query every 2 seconds
```

1) Basics (Core SQL)

Databases, Schemas, Tables

```
-- Create database and connect
CREATE DATABASE appdb;
-- (Then in shell) psql -d appdb

-- Schemas
CREATE SCHEMA app AUTHORIZATION app_user;
SET search_path TO app, public;

-- Tables
CREATE TABLE app.users (
  id          BIGSERIAL PRIMARY KEY,
  email       TEXT NOT NULL UNIQUE,
  full_name   TEXT,
  created_at  TIMESTAMPTZ DEFAULT now(),
  is_active   BOOLEAN DEFAULT TRUE
);
-- Alter / Drop
```

```
ALTER TABLE app.users ADD COLUMN last_login TIMESTAMPTZ;
DROP TABLE IF EXISTS app.users CASCADE;
```

Data Types (Common)

• `boolean`, `smallint`, `integer`, `bigint`, `numeric(p,s)`, `real`, `double precision`
• `text`, `varchar(n)`, `char(n)`
• `date`, `time`, `timestamp`, `timestamptz` (timestamp with time zone)
• `uuid`, `json`, `jsonb`, `bytea`, `inet`, `cidr`, `macaddr`, `interval`
• Arrays: `integer[]`, `text[]`

Insert, Update, Delete, Upsert

```
INSERT INTO app.users (email, full_name)
VALUES ('a@x.com', 'Alice'), ('b@x.com', 'Bob');

UPDATE app.users
SET last_login = now()
WHERE email = 'a@x.com';

DELETE FROM app.users WHERE is_active = FALSE;

-- Upsert (INSERT ... ON CONFLICT)
INSERT INTO app.users (email, full_name)
VALUES ('a@x.com', 'Alicia')
ON CONFLICT (email) DO UPDATE
SET full_name = EXCLUDED.full_name,
    updated_at = now();
```

Select, Filter, Sort, Limit

```
SELECT id, email, created_at
FROM app.users
WHERE is_active
ORDER BY created_at DESC
LIMIT 10 OFFSET 20;
```

Aggregation & Grouping

```
SELECT is_active, COUNT(*) AS users
FROM app.users
GROUP BY is_active
HAVING COUNT(*) > 0
ORDER BY users DESC;
```

Joins & Set Operations

```
-- Example tables
CREATE TABLE app.teams (
  id  BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL UNIQUE
);
CREATE TABLE app.memberships (
  user_id BIGINT REFERENCES app.users(id),
  team_id BIGINT REFERENCES app.teams(id),
  PRIMARY KEY (user_id, team_id)
);

-- JOINs
SELECT u.email, t.name AS team
FROM app.users u
JOIN app.memberships m ON m.user_id = u.id
JOIN app.teams t ON t.id = m.team_id
WHERE u.is_active;
-- LEFT JOIN (keep users even without a team)
```

```
SELECT u.email, t.name AS team
FROM app.users u
LEFT JOIN app.memberships m ON m.user_id = u.id
LEFT JOIN app.teams t ON t.id = m.team_id;

-- UNION / INTERSECT / EXCEPT
SELECT email FROM app.users WHERE is_active
UNION
SELECT email FROM app.users WHERE last_login IS NOT NULL;
```

Constraints

```
-- Primary key, unique, foreign key, check
ALTER TABLE app.users
  ADD CONSTRAINT users_email_chk CHECK (position('@' IN email) > 1);
```

2) Intermediate

Transactions & Isolation

```
BEGIN;  -- or START TRANSACTION
UPDATE app.users SET is_active = FALSE WHERE last_login < now() - interval '1 year';
SAVEPOINT s1;
DELETE FROM app.memberships WHERE user_id = 42;
ROLLBACK TO SAVEPOINT s1;  -- undo last step, keep earlier changes
COMMIT;  -- or ROLLBACK

-- Isolation
SHOW default_transaction_isolation;  -- typically read committed
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

Indexes (Basics)

```
-- B-tree (default)
CREATE INDEX users_email_idx ON app.users (email);
-- Unique index (also enforces uniqueness)
CREATE UNIQUE INDEX users_email_uq ON app.users (email);
-- Expression index
CREATE INDEX users_lower_email_idx ON app.users ((lower(email)));
-- Partial index
CREATE INDEX users_active_idx ON app.users (created_at) WHERE is_active;
-- Covering index (INCLUDE)
CREATE INDEX users_created_at_include_email ON app.users (created_at) INCLUDE (email);
```

Explain & Analyze

```
EXPLAIN SELECT * FROM app.users WHERE email = 'a@x.com';
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
  SELECT * FROM app.users WHERE lower(email) = 'a@x.com';
```

Views & Materialized Views

```
CREATE VIEW app.active_users AS
SELECT * FROM app.users WHERE is_active;

CREATE MATERIALIZED VIEW app.recent_users AS
SELECT * FROM app.users WHERE created_at > now() - interval '7 days';

REFRESH MATERIALIZED VIEW CONCURRENTLY app.recent_users;  -- needs unique index on MV
```

CTEs (WITH) & Recursion

```
WITH latest AS (
  SELECT id, email, row_number() OVER (ORDER BY created_at DESC) AS rn
  FROM app.users
)
SELECT * FROM latest WHERE rn <= 10;

-- Recursive CTE (example hierarchy)
WITH RECURSIVE subteams AS (
  SELECT id, name, id AS root_id FROM app.teams
  UNION ALL
  SELECT t.id, t.name, s.root_id
  FROM app.teams t
  JOIN subteams s ON t.id = s.id  -- Replace with real parent-child when you have it
)
SELECT * FROM subteams;
```

Window Functions

```
SELECT
  email,
  created_at,
  row_number() OVER (ORDER BY created_at)                                  AS rn,
  lag(created_at)  OVER (ORDER BY created_at)                              AS prev_created,
  count(*)        OVER ()                                                  AS total,
  rank()          OVER (ORDER BY created_at DESC)                          AS recency_rank,
  avg(extract(epoch FROM now() - created_at)) OVER ()                      AS avg_age_seconds
FROM app.users;
```

JSON/JSONB & Arrays

```
-- JSONB operators
SELECT
  payload -> 'user'            AS user_obj,
  payload ->> 'status'         AS status_text,
  payload #> '{meta,tags}'     AS tags_array
FROM app.events;

-- Index for JSONB containment
CREATE INDEX events_payload_gin ON app.events USING gin (payload jsonb_path_ops);
SELECT * FROM app.events WHERE payload @> '{"status":"ok"}';

-- Arrays & unnest
SELECT id, unnest(preferences) AS pref FROM app.users;  -- preferences is text[]
```

Sequences & Identity

```
CREATE TABLE app.items (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL
);
```

COPY (Fast I/O)

```
-- Server-side (needs server file access)
COPY app.users (email, full_name)
FROM '/var/lib/postgresql/import.csv' CSV HEADER;

-- Client-side (psql) – uses your local filesystem
\copy app.users (email, full_name) FROM './import.csv' CSV HEADER
\copy (SELECT * FROM app.users) TO './export.csv' CSV HEADER
```

3) Advanced

Table Partitioning

```
-- Range partitioning by created_at (monthly)
CREATE TABLE app.logs (
  id          BIGSERIAL,
  created_at  timestamptz NOT NULL,
  payload     jsonb,
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE TABLE app.logs_2025_01 PARTITION OF app.logs
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

CREATE TABLE app.logs_default PARTITION OF app.logs DEFAULT;  -- catch-all
```

Advanced Index Types

```
-- GIN for JSONB/arrays/FTS
CREATE INDEX users_prefs_gin ON app.users USING gin (preferences);
-- GiST for geometric/range types
CREATE INDEX ranges_gist ON app.ranges USING gist (period);
-- BRIN for very large, naturally ordered tables
CREATE INDEX logs_brin ON app.logs USING brin (created_at);
```

Full-Text Search (FTS)

```
ALTER TABLE app.docs ADD COLUMN tsv tsvector;
UPDATE app.docs SET tsv = to_tsvector('english', coalesce(title,'') || ' ' || coalesce(body,''));
CREATE INDEX docs_tsv_idx ON app.docs USING gin (tsv);

SELECT id, ts_rank_cd(tsv, plainto_tsquery('english', 'postgres tutorial')) AS rank
FROM app.docs
WHERE tsv @@ plainto_tsquery('english', 'postgres tutorial')
ORDER BY rank DESC;
```

Concurrency & Locks

```
-- Inspect locks
SELECT * FROM pg_locks l JOIN pg_stat_activity a USING (pid);

-- Row-level locks
SELECT * FROM app.users WHERE id = 42 FOR UPDATE SKIP LOCKED;  -- or NOWAIT
```

Security: Roles, Privileges, RLS

```
-- Roles & privileges
CREATE ROLE app_user LOGIN PASSWORD 'change_me';
GRANT USAGE ON SCHEMA app TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO app_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA app
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_user;

-- Row-Level Security
ALTER TABLE app.users ENABLE ROW LEVEL SECURITY;
CREATE POLICY users_is_self ON app.users
  USING (current_user = email);  -- simplistic example if user == email
```

Extensions (Common)

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pg_trgm;        -- trigram search
CREATE EXTENSION IF NOT EXISTS uuid-ossp;      -- UUID generation
```

```
CREATE EXTENSION IF NOT EXISTS hstore;         -- key-value
```

Performance: Stats & Tuning

```
-- What is running
SELECT pid, usename, state, query, now() - query_start AS runtime
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY runtime DESC;

-- Query stats (after enabling extension)
SELECT * FROM pg_stat_statements ORDER BY total_time DESC LIMIT 20;

-- Manual maintenance
VACUUM (ANALYZE) app.users;
REINDEX TABLE app.users;           -- rebuild corrupted/bloated index
CLUSTER app.users USING users_email_idx;  -- reorder table by index (blocking)
```

Backup & Restore

```
# Logical backups
pg_dump -Fc -f appdb.dump appdb                # custom format
pg_restore -d appdb_restored appdb.dump        # restore
pg_dump -Fd -j 4 -f appdb_dir appdb            # directory format, parallel

# Plain SQL (easiest to inspect)
pg_dump appdb > appdb.sql
psql -d appdb_restored -f appdb.sql
```

Logical Replication (Basics)

```
-- On publisher
ALTER SYSTEM SET wal_level = logical;  -- requires restart
SELECT pg_reload_conf();
CREATE PUBLICATION pub_app FOR TABLE app.users, app.teams;

-- On subscriber
CREATE SUBSCRIPTION sub_app
  CONNECTION 'host=... dbname=appdb user=replicator password=...'
  PUBLICATION pub_app;
```

4) Expert

Planner Mastery

```
-- Inspect different plan shapes
SET enable_nestloop = off;  -- force hash/merge joins
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;

-- Prepared statements and plan cache
PREPARE q1(text) AS SELECT * FROM app.users WHERE email = $1;
EXECUTE q1('a@x.com');
DEALLOCATE q1;
```

Parallel Query Controls

```
SET max_parallel_workers_per_gather = 4;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM app.logs;  -- look for Gather nodes
```

Foreign Data Wrappers (FDW)

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
CREATE SERVER remotedb FOREIGN DATA WRAPPER postgres_fdw OPTIONS (host '10.0.0.5', dbname 'other');
CREATE USER MAPPING FOR app_user SERVER remotedb OPTIONS (user 'remote', password 'secret');
IMPORT FOREIGN SCHEMA public FROM SERVER remotedb INTO remote;
SELECT * FROM remote.some_table LIMIT 10;
```

Row-Level Auditing (example)

```
CREATE TABLE app.audit_users (
  at         timestamptz DEFAULT now(),
  who        text,
  action     text,
  old_row    jsonb,
  new_row    jsonb
);

CREATE OR REPLACE FUNCTION app.audit_users_fn()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO app.audit_users(who, action, new_row)
    VALUES (current_user, 'INSERT', to_jsonb(NEW));
  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO app.audit_users(who, action, old_row, new_row)
    VALUES (current_user, 'UPDATE', to_jsonb(OLD), to_jsonb(NEW));
  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO app.audit_users(who, action, old_row)
    VALUES (current_user, 'DELETE', to_jsonb(OLD));
  END IF;
  RETURN NULL;
END
$$;

DROP TRIGGER IF EXISTS audit_users_trg ON app.users;
CREATE TRIGGER audit_users_trg
AFTER INSERT OR UPDATE OR DELETE ON app.users
FOR EACH ROW EXECUTE FUNCTION app.audit_users_fn();
```

Event Triggers (Schema-change hooks)

```
CREATE OR REPLACE FUNCTION app.on_ddl_end() RETURNS event_trigger LANGUAGE plpgsql AS $$
BEGIN
  RAISE NOTICE 'DDL finished: %', tg_tag;
END;$$;

CREATE EVENT TRIGGER ddl_end_trigger ON ddl_command_end
EXECUTE PROCEDURE app.on_ddl_end();
```

Security Hardening (Pointers)

• Use SCRAM-SHA-256 passwords: `password_encryption = 'scram-sha-256'`
• Enforce least privilege (separate app role vs. migration/owner role)
• Enable SSL/TLS for remote connections
• Consider `pgaudit` extension for auditable environments

5) `psql` Power-User Moves

```
\e              -- open editor for current query
\gexec          -- execute query result as commands
\p              -- show the query buffer
\r              -- reset/clear the query buffer
\a / \H         -- toggle unaligned & HTML output
\o file.txt     -- send query output to a file
\encoding UTF8  -- set client encoding
```

```
\set             -- list or set variables
```

6) Hands-on Mini-Labs (Practice)

Lab A — Build & Query

```
CREATE SCHEMA lab;
CREATE TABLE lab.sales (
  id          bigserial PRIMARY KEY,
  ts          timestamptz NOT NULL DEFAULT now(),
  region      text NOT NULL,
  product     text NOT NULL,
  amount      numeric(12,2) NOT NULL
);

INSERT INTO lab.sales(region, product, amount)
SELECT (ARRAY['APAC','EMEA','AMER'])[ceil(random()*3)],
       (ARRAY['A','B','C'])[ceil(random()*3)],
       round((random()*1000)::numeric, 2)
FROM generate_series(1, 10000);

-- Queries
SELECT region, sum(amount) FROM lab.sales GROUP BY region ORDER BY 2 DESC;
CREATE INDEX sales_ts_idx ON lab.sales(ts);
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM lab.sales WHERE ts > now() - interval '1 day';
```

Lab B — JSONB & GIN

```
CREATE TABLE lab.events (
  id bigserial PRIMARY KEY,
  payload jsonb NOT NULL
);

INSERT INTO lab.events(payload)
VALUES ('{"type":"click","user":"u1","meta":{"tags":["home","cta"]}}'),
       ('{"type":"view","user":"u2","meta":{"tags":["product","specs"]}}');

CREATE INDEX events_payload_gin ON lab.events USING gin (payload);
SELECT * FROM lab.events WHERE payload @> '{"type":"click"}';
```

Lab C — Partitioning

```
CREATE TABLE lab.logs (
  id bigserial,
  ts timestamptz not null,
  msg text,
  primary key (id, ts)
) PARTITION BY RANGE (ts);

CREATE TABLE lab.logs_2025_01 PARTITION OF lab.logs
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

INSERT INTO lab.logs(ts,msg)
SELECT now() - (g * interval '1 hour'), 'hello '||g
FROM generate_series(1, 100) g;
```

7) Performance Checklist (Quick Reference)

1. **Find slow queries:** `pg_stat_statements` (top total_time, mean_time)
2. **Get plans:** `EXPLAIN (ANALYZE, BUFFERS)`; confirm right indexes are used

3. **Indexes:** cover predicates, avoid over-indexing; consider partial/expression/GIN/BRIN
4. **Stats:** `ANALYZE` after major changes; adjust `default_statistics_target` if needed
5. **Memory:** tune `work_mem` for sorts/joins; `maintenance_work_mem` for maintenance
6. **IO:** check `shared_buffers`, `effective_cache_size`; monitor cache hit ratio
7. **Bloat:** `VACUUM`, `VACUUM (FULL)` (blocking) or external tools; consider `REINDEX`
8. **Locking:** inspect `pg_locks`, use `NOWAIT`/`SKIP LOCKED`; reduce transaction scope
9. **Partitioning:** for very large tables by time/id ranges; prune old partitions
10. **Concurrency:** test real workloads; consider parallelism and connection pooling

8) Useful System Catalogs & Views

```
-- Activity, locks, index usage
SELECT * FROM pg_stat_activity;
SELECT * FROM pg_locks;
SELECT * FROM pg_stat_user_indexes;
SELECT * FROM pg_stat_user_tables;
```

9) Learning Roadmap (Suggested Path)

• **Week 1:** psql basics, DDL/DML, SELECT + JOINs, constraints
• **Week 2:** transactions, indexes, EXPLAIN, views/materialized views, CTEs
• **Week 3:** window functions, JSONB & arrays, COPY, sequences/identity
• **Week 4:** partitioning, FTS, monitoring (pg_stat_*), VACUUM/ANALYZE
• **Week 5+:** RLS & security, extensions, replication basics, FDW, PL/pgSQL, event triggers, parallelism, advanced tuning

> Tip: Keep a journal of plans and timings as you tune. Regressions are easier to spot with history.

Disclaimer

Use caution when running `DROP`, `ALTER`, `VACUUM (FULL)`, `CLUSTER`, or changes to replication/`ALTER SYSTEM` in production. Always test in staging and ensure backups.